

OAI 5G RFSim — Complete Setup & Run Documentation

A full record of bringing up an end-to-end 5G network (gNB + UE + 5G Core) in the RF simulator using Docker, with every command and option explained, the actual control-plane communication that occurred, and the commands used to observe it.

Repo: `openairinterface5g` cloned under `~/OAI_Begineers_Experiment/`

Deployment dir: `ci-scripts/yaml_files/5g_rfsimulator/`

Result: UE 208990100001100 registered (5GMM-REGISTERED), got IP 12.1.1.2, pinged the data network with 0% loss.

Part A — Install Docker Engine + Compose plugin

The machine had no docker at all (`docker compose version` → "Command 'docker' not found"). Docker was installed from Docker's official APT repository (newer and more complete than Ubuntu's `docker.io`).

A.1 Add Docker's GPG key and repository

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

Command / option	Meaning
<code>install -m 0755 -d <dir></code>	Create the directory with permission mode 0755 (owner rwx, others r-x)
<code>curl -f</code>	Fail silently (no HTML error page written) on HTTP errors
<code>curl -s / -S</code>	Silent (no progress bar) but still Show errors
<code>curl -L</code>	Follow HTTP redirects
<code>curl -o <file></code>	Write output to a file instead of stdout
<code>chmod a+r</code>	Make the key readable by all users (apt runs unprivileged checks)

Then the repository definition was written (deb822 format), auto-filling the Ubuntu codename (noble) and architecture (amd64):

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
```


Option	Meaning
<code>usermod -a -G docker</code>	Append the user to the supplementary Group docker (without -a, other groups would be removed)
<code>newgrp docker</code>	Start a subshell whose active group is docker, so the change takes effect immediately

After this, `docker ...` works without `sudo`.

A.5 Confirm the TUN device exists

```
ls /dev/net/tun          # expect: /dev/net/tun
```

The UE needs `/dev/net/tun` to create its `oaitun_ue1` data interface later. The compose file passes this device into the UE container.

Part B — Pre-flight checks on the clone

Before deploying, the files the run depends on were confirmed present, and the subscriber data was inspected.

```
cd ~/OAI_Begineers_Experiment/openairinterface5g
ls ci-scripts/yaml_files/5g_rfsimulator/          # docker-compose.yaml, oai_db.sql,
mini_nonrf_config.yaml, ...
ls ci-scripts/conf_files/nrue.uicc.conf           # the UE's SIM config
```

B.1 Which subscriber table does the seed DB use?

```
grep -iE "create table|insert into" \
  ci-scripts/yaml_files/5g_rfsimulator/oai_db.sql | head -30
```

Option	Meaning
<code>grep -i</code>	Case-insensitive match
<code>grep -E</code>	Extended regex (enables the <code> </code> alternation)
<code>head -30</code>	First 30 lines only

Observed: the DB uses a legacy `users` table (plus 4G-era `pdn`, `mmeidentity`, etc.) — not `AuthenticationSubscription/SessionManagementSubscription`. This matters for adding custom UEs later: in this deployment, subscriber rows go in the `users` table.

B.2 Confirm the UE SIM matches the DB

```
grep -E "imsi|key|opc|dnn" ci-scripts/conf_files/nrue.uicc.conf
```

```
grep -i "2089901000011" ci-scripts/yaml_files/5g_rfsimulator/oai_db.sql
```

Observed — they match:

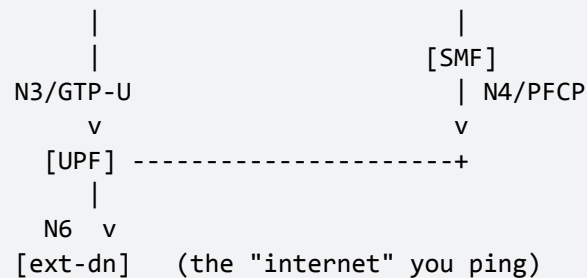
Field	UE (nrue.uicc.conf)	DB (users row)
IMSI/SUPI	208990100001100	208990100001100
Key (Ki)	fec86ba6eb707ed08905757b1bb44b8f	0xfec86ba6eb707ed08905757b1bb44b8f
OPc	C42449363BBAD02B66D16BC975D77CC1	0xc42449363bbad02b66d16bc975d77cc1
DNN / slice	oai / sst 1	(allowed in core config)

Matching credentials are the precondition for 5G-AKA authentication to succeed.

Part C — Deploying the network (in the correct order)

Topology of what gets deployed:

```
[UE] --rfsim(I/Q over TCP)--> [gNB] --N2/NGAP(SCTP)--> [AMF] --- [MySQL]
(subscribers)
```



```
Networks: rfsim5g-oai-public-net = 192.168.71.0/24 (RAN <-> core signaling)
          rfsim5g-oai-traffic-net = 192.168.72.0/24 (UPF <-> data network)
```

All commands run from ci-scripts/yaml_files/5g_rfsimulator/.

Step 1 — Start the 5G Core first

```
docker compose up -d mysql oai-amf oai-smf oai-upf oai-ext-dn
```

Option	Meaning
up	Create + start the named services
-d	Detached: run in the background, return the prompt
(service list)	Start only these 5 — the core must be healthy before the gNB

Compose pulled the images on first run: mysql:9.6, oaisoftwarealliance/oai-amf:v2.2.1, oai-smf:v2.2.1, oai-upf:v2.2.1, trf-gen-cn5g:latest, and created the two networks.

Observe health — repeat until all show (healthy):

```
docker compose ps -a
```

Option	Meaning
ps	List this project's containers
-a	Include stopped/all containers, not just running

The containers progressed health: starting → healthy over ~60s (MySQL slowest — it seeds from oai_db.sql). Final state:

```
rfsim5g-mysql      Up (healthy)
rfsim5g-oai-amf    Up (healthy)
rfsim5g-oai-smf    Up (healthy)
rfsim5g-oai-upf    Up (healthy)
rfsim5g-oai-ext-dn Up (healthy)
```

Step 2 — Start the gNB

```
docker compose up -d oai-gnb      # pulls oaisoftwarealliance/oai-gnb:develop
sleep 10
```

Observe the gNB↔AMF connection (NGSetup):

```
docker logs rfsim5g-oai-amf 2>&1 | grep -A 6 "gNBs"
```

Option	Meaning
docker logs <c>	Print a container's stdout/stderr
2>&1	Merge stderr into stdout so grep sees everything
grep -A 6	Print the matching line + 6 lines After it

Early snapshots showed an empty table (gNB not yet connected):

	Index		Status		Global Id		gNB Name		PLMN	
	-		-		-		-		-	

then, once NGSetup completed:

	1		Connected		0x0E00		gnb-rfsim		208,99	
--	---	--	-----------	--	--------	--	-----------	--	--------	--

The AMF prints this table on a timer, so the - rows are simply snapshots taken before the gNB associated. Connected = the SCTP/NGAP link is up and the gNB and AMF agreed on PLMN 208/99. This happens once at startup, before any UE.

Step 3 — Start the UE

```
docker compose up -d oai-nr-ue      # pulls oaisoftwarealliance/oai-nr-ue:develop
sleep 20                             # allow registration to complete
```

The UE container's options come from the compose file's USE_ADDITIONAL_OPTIONS: -E --rfsim -r 106 --numerology 1 -C 3619200000 --rfsimulator.[0].serveraddr <gNB IP>.

UE option	Meaning
-E	Continuous transmission (suits the single-antenna rfsim profile)
--rfsim	Use the RF simulator device instead of a real SDR
-r 106	106 PRBs of bandwidth (~40 MHz at 30 kHz SCS)
--numerology 1	Subcarrier spacing index 1 = 30 kHz
-C 3619200000	Downlink carrier frequency in Hz (must match the gNB SSB)
--rfsimulator.[0].serveraddr <ip>	Connect the rfsim socket to the gNB

Part D — The communication that happened

D.1 gNB ↔ AMF: NGSetup (control-plane bring-up)

Before any UE, the gNB opened an SCTP association to the AMF on port 38412 and exchanged NGSetupRequest / NGSetupResponse. Evidence: the AMF's gNB table flipping to Connected (Part C, Step 2). This is the N2 link.

D.2 UE registration + authentication + PDU session

When the UE started, the AMF log captured the full sequence (all timestamped 07:06:34 UTC — the whole exchange took ~100 ms). Message ladder:

UE	Network (gNB relay -> AMF + SMF/UPF/DB)
--- RRC setup + Registration Request ----->	
<--- Authentication Request ----->	5G-AKA
--- Authentication Response ----->	
<--- Security Mode Command ----->	ciphering on
--- Security Mode Complete ----->	
<--- Registration Accept (+ GUTI) ----->	
--- Registration Complete ----->	=> 5GMM-REGISTERED
--- PDU Session Establishment Request ----->	
<--- PDU session setup (builds GTP-U tunnel) ----->	=> IP 12.1.1.2

Mapped to the actual AMF log lines:

Step	Log evidence (from rfsim5g-oai-amf)	Meaning
Registration Request	Received Registration Request message, handling...; state 5GMM-DEREGISTERED -> COMM-PROC-INIT (REGISTRATION_REQUEST_RECEIVED)	UE asks to register; AMF starts the procedure
Authentication	Received Security Vectors, try to setup security; AUTHENTICATION_REQUEST_SENT; AUTHENTICATION_RESPONSE_RECEIVED	5G-AKA challenge/response; passes because Key/OPc match the DB
Security	Start Security Mode Control procedure; SECURITY_MODE_COMMAND_SENT; Received Security Mode Complete	NAS ciphering + integrity activated
Registration done	REGISTRATION_ACCEPT_SENT_WITH_T3550; Received Registration Complete; State 5GMM-REGISTERED ... successfully updated!	UE is registered; assigned GUTI 20899010041395994520
PDU session	Running ITTI_SMF_PDU_SESSION_CREATE_SM_CTX; Handle PDU Session Establishment Request; PDU_RES_SETUP_REQ; Handle PDU Session Resource Setup Response	AMF→SMF create session; SMF programs UPF; gNB GTP-U tunnel set up

Notes on reading these logs:

5GMM = 5G Mobility Management state machine. The UE moves DEREGISTERED → COMM-PROC-INIT → 5GMM-REGISTERED.

GUTI = temporary identity issued so the permanent IMSI isn't repeatedly sent over the air.

RAN UE NGAP ID / AMF UE NGAP ID = 0x01 = the per-UE connection identifiers on the N2 interface.

Free NGAP Message PDU lines are just memory cleanup after each NGAP message — ignorable noise.

Part E — Observability: which command shows what

To observe...	Command
Docker daemon running	<code>sudo systemctl status docker</code>
Images + container health/state	<code>docker compose ps -a</code>
Which images/volumes the deploy uses	<code>grep -nE "image: volumes:" docker-compose.yaml</code>
gNB↔AMF (NGSetup) connection	<code>docker logs rfsim5g-oai-amf 2>&1 grep -A 6 "gNBs"</code>
Full registration / auth sequence	<code>docker logs rfsim5g-oai-amf 2>&1 grep -E "Registration PDU 5GMM Security"</code>
gNB-side radio/RRC view	<code>docker logs rfsim5g-oai-gnb</code>
UE-side sync/RRC/NAS view	<code>docker logs rfsim5g-oai-nr-ue</code>

To observe...	Command
UE got an IP (success signal)	<code>docker exec -it rfsim5g-oai-nr-ue ip addr show oaitun_ue1</code>
A container's IP address	<code>docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' <container></code>
Live packet capture (handshake)	<code>sudo tcpdump -i rfsim5g-public -w /tmp/oai.pcap (open in Wireshark; filter ngap nas-5gs pfcf gtp)</code>

Option	Meaning
<code>docker exec -it</code>	Run a command inside a running container; -i interactive, -t allocate a TTY
<code>ip addr show <iface></code>	Show one network interface's addresses
<code>docker inspect -f '<go-template>'</code>	Print specific fields using a Go template format string

Part F — Data-plane test (prove user traffic flows)

```
# Discover the data network container's IP on the traffic net
EXTDN=$(docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' \
    rfsim5g-oai-ext-dn | tr ' ' '\n' | grep '192.168.72')
echo "ext-dn = $EXTDN"          # -> 192.168.72.135

# Ping it THROUGH the 5G tunnel interface
docker exec -it rfsim5g-oai-nr-ue ping -I oaitun_ue1 -c 5 "$EXTDN"
```

Option	Meaning
<code>ping -I oaitun_ue1</code>	Send via the Interface oaitun_ue1 (forces traffic through the 5G tunnel)
<code>ping -c 5</code>	Send exactly 5 packets then stop
<code>tr ' ' '\n'</code>	Translate spaces to newlines (so grep can filter one IP per line)

Observed result:

```
5 packets transmitted, 5 received, 0% packet loss
rtt min/avg/max/mdev = 3.554/4.901/6.123/0.918 ms
```

The path was: UE → oaitun_ue1 → gNB (over rfsim) → GTP-U tunnel → UPF → ext-dn. 0% loss = the full user plane works.

Part G — Teardown

```
docker compose down          # stop and remove containers + networks
docker compose down -v      # also remove the MySQL volume (use if you edit
oai_db.sql)
```

Option	Meaning
down	Stop and remove this project's containers and networks
-v	Also delete named volumes (forces MySQL to re-seed from oai_db.sql next time)

Appendix — End-to-end run, condensed

```
# 0. one-time: docker installed + user added to docker group (Part A)
cd ~/OAI_Beginners_Experiment/openairinterface5g/ci-scripts/yaml_files/5g_rfsimulator

# 1. core first, wait for healthy
docker compose up -d mysql oai-amf oai-smf oai-upf oai-ext-dn
watch -n2 docker compose ps -a          # Ctrl-C when all (healthy)

# 2. gNB, confirm NGSetup
docker compose up -d oai-gnb && sleep 10
docker logs rfsim5g-oai-amf 2>&1 | grep -A 6 "gNBs"      # expect "Connected"

# 3. UE, confirm registration + IP
docker compose up -d oai-nr-ue && sleep 20
docker exec -it rfsim5g-oai-nr-ue ip addr show oaitun_ue1  # expect inet 12.1.1.2

# 4. data-plane test
docker exec -it rfsim5g-oai-nr-ue ping -I oaitun_ue1 -c 5 192.168.72.135

# 5. teardown
docker compose down
```

Key facts for this deployment

- PLMN 208/99; default UE IMSI 208990100001100; UE IP pool 12.1.1.0/24.
- Subscriber table in oai_db.sql is users (legacy schema) — that's where a custom UE row goes here.
- Core is the minimal no-NRF set (MySQL + AMF + SMF + UPF + ext-dn); the AMF reads subscriber data directly from MySQL.
- Carrier -C 3619200000 Hz, 106 PRBs, 30 kHz SCS, band 78 (TDD).